Atividade Prática – Aula 3

Modelagem com e sem Estrutura no MongoDB

Objetivo:

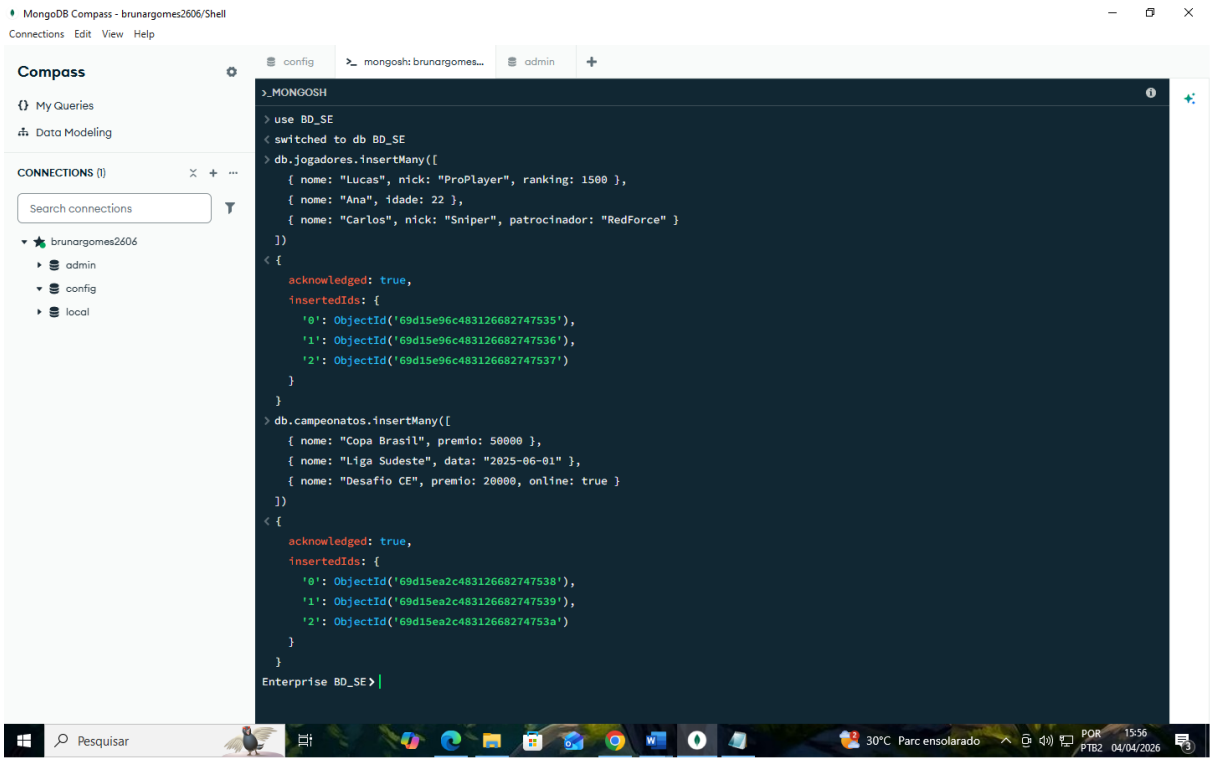
• Flexibilidade

• Controle estrutural

• Comparação prática

• Raciocínio crítico

EXERCÍCIO 1 – Banco Sem Estrutura Definida



Reflexão

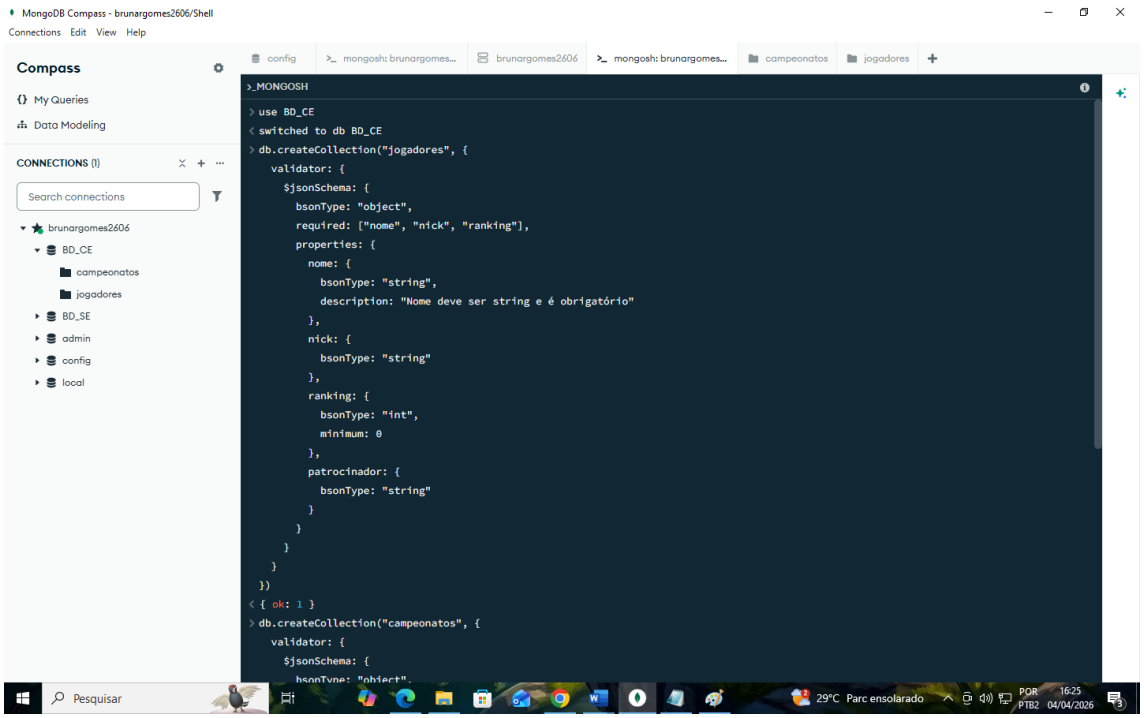
1. Todos os documentos seguem um padrão?
   Não. Cada documento possui campos diferentes (alguns têm nick, outros idade, outros patrocinador).

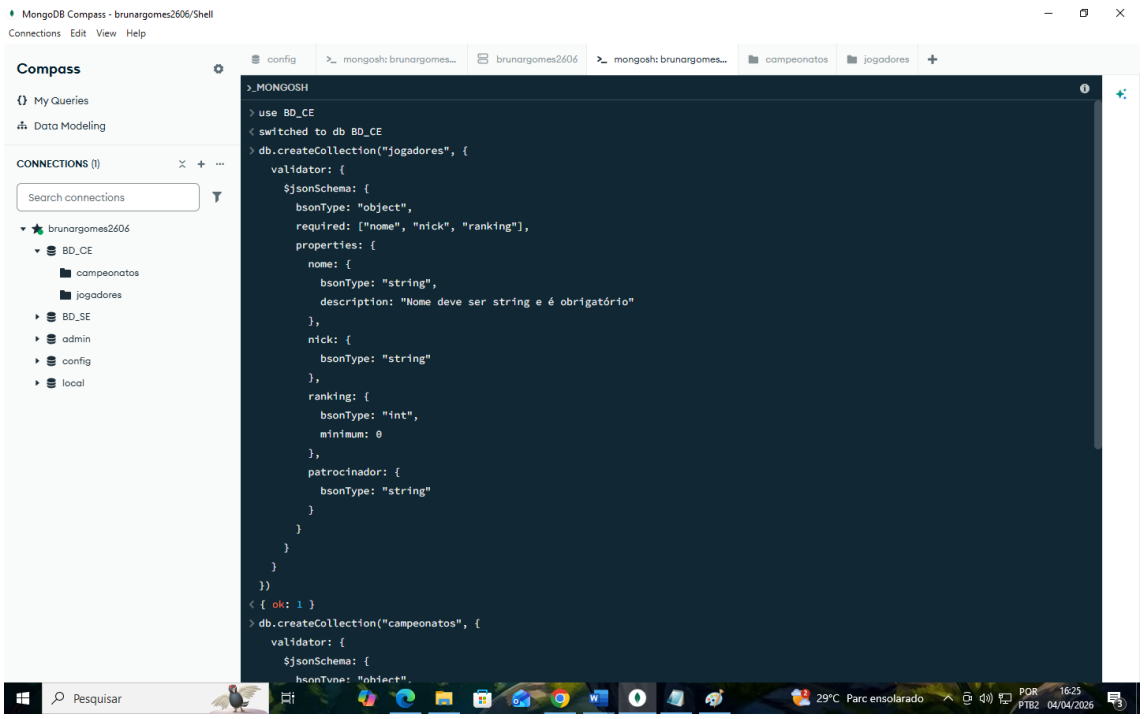
2. Isso pode gerar problema em sistemas grandes?
   Sim. Se uma consulta tentar filtrar por "ranking", por exemplo, o primeiro documento (que não tem esse campo) simplesmente não aparecerá no resultado — sem aviso de erro. Isso pode gerar relatórios incompletos ou análises incorretas, já que o

desenvolvedor pode não perceber que parte dos dados está ausente. Além disso, campos com nomes escritos de forma diferente (ex.: "redeSocial" vs "rede_social") tratam-se como campos distintos, fragmentando ainda mais os dados.

3. Como o sistema sabe o que é obrigatório?
   Não sabe. No modelo sem estrutura, não existe validação automática, então qualquer dado pode ser inserido.

EXERCÍCIO 2 – Banco Com Estrutura Definida





Teste de Inserção Válida

```
> db.jogadores.insertOne({
    nome: "Lucas",
    nick: "ProPlayer",
    ranking: 1500
  })
< {
    acknowledged: true,
    insertedId: ObjectId('69d165fd4ababde40c12c8d9')
  }
Enterprise BD_CE >
```

Teste de Inserção Inválida

```
> db.jogadores.insertOne({
    nome: "Ana"
  })
✖ ▸ MongoServerError: Document failed validation
Enterprise BD_CE >
```

ATIVIDADE AVALIATIVA – Aula 3

Tarefa 1 – Inserção Livre

Insira:

- Um jogador com apenas nome
- Um jogador com nome, ranking e patrocinador
- Um jogador com um campo inesperado chamado “redeSocial”

```
>_MONGOSH
> use BD_SE
< switched to db BD_SE
> db.jogadores.insertOne({
    nome: "Paulo"
  })
< {
    acknowledged: true,
    insertedId: ObjectId('69d2c3e791c10d312a181f89')
  }
> db.jogadores.insertOne({
    nome: "Carla",
    nick: "CarlaZ",
    ranking: "mil e quinhentos"
  })
< {
    acknowledged: true,
    insertedId: ObjectId('69d2c46491c10d312a181f8a')
  }
> db.jogadores.insertOne({
    nome: "Diego",
    nick: "DarkShot",
    ranking: 980,
    redeSocial: "@darkshot_gamer"
  })
< {
    acknowledged: true,
    insertedId: ObjectId('69d2c4b791c10d312a181f8b')
  }
Enterprise BD_SE>
```

Tarefa 2 – Análise Crítica

1. Todos os documentos possuem o mesmo padrão?
   Não. No BD_SE cada documento foi inserido com uma estrutura completamente diferente, e o MongoDB aceitou todos sem reclamar. Não existe nenhum campo obrigatório, nenhum tipo definido e nenhuma regra de consistência.

2. Isso pode gerar problemas em consultas futuras?
   Sim. Quando uma consulta filtra por um campo que só existe em parte dos documentos, os demais simplesmente desaparecem do resultado — sem erro, sem aviso. O desenvolvedor pode acreditar que os dados estão completos quando na verdade estão incompletos.

3. Cite um risco desse modelo em sistemas grandes.
   Corrupção silenciosa dos dados em escala. Em um sistema com muitos desenvolvedores inserindo dados, pequenas inconsistências se acumulam: campos com nomes escritos de formas diferentes (redeSocial vs rede_social), tipos misturados (ranking como número em um documento e como string em outro) e campos obrigatórios ausentes. Com milhares de documentos, identificar e corrigir essas inconsistências se torna extremamente custoso e pode comprometer relatórios, integrações com APIs e a confiabilidade do sistema como um todo.

PARTE 2 – Banco Com Estrutura (BD_CE)

Tarefa 3 – Inserção Correta

Insira um jogador válido, obedecendo o schema.

```
>_MONGOSH
> use BD_CE
< switched to db BD_CE
> db.jogadores.insertOne({
    nome: "Mariana",
    nick: "MariFire",
    ranking: NumberInt(1750)
  })
< {
    acknowledged: true,
    insertedId: ObjectId('69d2c665915eb2079425beed')
  }
Enterprise BD_CE>
```

Tarefa 4 – Inserção Inválida

Tente inserir um jogador:

- Sem campo obrigatório ou com ranking como string

```
>_MONGOSH
> use BD_CE
< switched to db BD_CE
> db.jogadores.insertOne({
    nome: "Mariana",
    nick: "MariFire",
    ranking: NumberInt(1750)
  })
< {
    acknowledged: true,
    insertedId: ObjectId('69d2c665915eb2079425beed')
  }
> db.jogadores.insertOne({
    nome: "Paulo"
  })
✖ ▸ MongoServerError: Document failed validation
Enterprise BD_CE>
```

Tarefa 5 – Comparação Técnica

Responda:

1. Qual banco oferece mais segurança para produção?
   O BD_CE oferece mais segurança para produção. A validação por jsonSchema garante que todos os documentos inseridos respeitam um contrato de dados: campos obrigatórios sempre presentes, tipos corretos e valores dentro dos limites definidos (ex.: ranking >= 0). Isso evita dados corrompidos, inconsistentes ou incompletos chegando ao ambiente produtivo.

2. Qual banco oferece mais flexibilidade para prototipação?
   O BD_SE oferece maior flexibilidade para prototipação. Como não há schema definido, a equipe pode inserir documentos com qualquer estrutura imediatamente, sem precisar definir previamente todos os campos. Isso acelera a experimentação nas fases iniciais do projeto, quando os requisitos ainda estão sendo descobertos.

3. Em qual cenário você utilizaria cada modelo?

- BD_SE: fases de exploração e prototipação rápida, hackathons, MVPs e ambientes de desenvolvimento onde a estrutura dos dados ainda está sendo definida e muda constantemente.

- BD_CE: sistemas em produção, APIs públicas ou integradas a outros sistemas, ambientes com múltiplos desenvolvedores inserindo dados, e qualquer cenário onde a integridade e consistência dos dados são requisitos críticos do negócio.

PARTE 3 – Raciocínio de Arquitetura

Situação-problema (estilo ENADE):

Uma empresa de jogos está crescendo rapidamente e pretende integrar seu banco com uma API pública. O sistema será acessado por milhares de usuários simultaneamente.

Com base nisso: Você recomendaria BD_SE ou BD_CE? Justifique tecnicamente.

O BD_CE é a recomendação correta para este cenário. Com milhares de usuários simultâneos e integração com API pública, o schema validado garante que todos os documentos sigam a mesma estrutura — evitando dados corrompidos e contratos de API quebrados. Em sistemas grandes, a falta de validação do BD_SE gera inconsistências silenciosas que se tornam impossíveis de corrigir sem downtime.